

Förord

Det är många som hjälpt oss med att påpeka misstag och förbättra texten. Särskilda insatser har gjorts av

- Rikard Erngren
- Samuel Lundqvist
- Evan Cavallo, som översatt till engelska!
- Lars Lidvall

Tack alla!

Kapitel 1

Kom igång med Python

1.1 Programmering och Python

Detta kompendium handlar om *programmering*, det vill säga att skriva program som en dator sedan kan köra för att lösa olika typer av problem. I kompendiet använder vi programmeringsspråket Python, men många av de underliggande idéerna går att överföra till andra språk (t.ex. Java, JavaScript, C/C++, Ruby, Scala, Haskell, OCaml). Python är ett väldigt populärt programmeringsspråk (testa att Googla “*most popular programming languages*”) som kan användas att skriva mjukvara till i princip alla sorters tillämpningar. Det används på många av de största IT-företagen, som Instagram, Google, YouTube, Spotify, och Amazon.

Vi kommer att använda version 3.8, eller senare, av Python. Vi rekommenderar även en installation av *Anaconda*, ett verktyg för att hantera paket och omgivningar för Python. Installationsinstruktioner finns på kurshemsidan samt genom en videotutorial som ligger på YouTube.

En *programmeringseditor* stödjer “syntax highlighting” och operationer som gör det lättare att programmera. Vi kommer att använda oss av *Spyder* som följer med installationen av *Anaconda* i denna kurs.

Exempel på andra programmeringseditorer:

- MS Visual Studio Code — gratis och idag mycket spridd. Du har växtmån.
- PyCharm — stark integration med Python
- För entusiaster (kan vara svåra att komma igång med):
 - Emacs
 - Vim

1.1.1 Att programmera i Python

När man programmerar i Python skriver man kod i en `.py`-fil. Denna kod laddas sedan i Python-tolken (eng. *interpreter*).

Startar man Spyder-applikationen inifrån Anaconda Navigator får man ett fönster med tre paneler: ‘editor’, ‘help’, och ‘console’. I kompendiet är kod som är skriven i editorn inramad med grå bakgrund. Vi kan till exempel skriva följande tre rader i Spyders editor:

```
print(17)
print(1.1)
print('DA2005')
```

Om vi trycker på den gröna triangeln uppe till vänster eller tangentbordskommando F5 tolkas denna kod i Python-tolken och skrivs ut i konsolen, rutan i Spyder (och liknande program) där man kan evaluera Python-uttryck. Om man inte sparat filen så dyker det först upp en ruta med “Save file”. Spara filen på valfritt ställe på hårddisken, men kom ihåg att ha bra struktur så att du hittar filerna senare. Ett smidigt sätt är att skapa mappen “DA2005” och ha alla lämpliga undermappar (t.ex. “labb 1”) där man sparar filerna för varje del av kursen.

Vi döper filen till `f1.py` och när vi kör den får vi följande i konsolen på macOS (det ser lite annorlunda ut på Linux och Windows):

```
runfile('/Users/kxs624/kurser/DA2005/f1.py',
        wdir='/Users/kxs624/kurser/DA2005/')
17
1.1
DA2005
```

Obs: i kompendiet är kod och resultat som skrivs ut i konsolen inramad med turkos bakgrund.

1.2 Stil i kompendiet

1.2.1 Kodrepresentation

Vi kommer att beskriva programmering med svenska termer, men när man programmerar är det bra att använda engelska för variabelnamn, kommentarer, dokumentation, och utskrifter. Att använda engelska är vedertagen praxis då man ofta jobbar i internationella miljöer.

När det är underförstått att vi skriver kod i texteditorn som vi vill exekvera, “köra”, och skriva ut i konsolen kommer vi visa den grå och turkosa miljön efter varandra på följande sätt:

```
print("welcome to DA2005")
print(2*2)
```

```
welcome to DA2005
4
```

1.2.2 Uppgifter

När det gäller uppgifter i kompendiet kommer vi att markera uppgifter som vi anser vara svårare med symbolen † först i uppgiftstexten.

1.3 Variabler

En av grundstenarna i programmering är *variabler*. En vanlig intuitiv modell för variabler är att se dem som lådor med namn som man kan stoppa in värden i.

Här är ett exempel där vi räknar ut medelhastigheten för en resa över 67 km på 1,25 timmar.

```
s = 67
t = 1.25
v = s / t
print(v)
```

Vi *tilldelar* värdet 67 till variabeln *s* och värdet 1,5 till variabeln *t*. Lägg märke till att decimalkomma måste skrivas på engelskt sätt, med en punkt. Därefter tilldelas variabeln *v* resultatet av beräkningen s / t . I Spyder, VS Code, eller motsvarande brukar man kunna se att variabeln *v* nu har värdet 53,6, men instruktionen `print(v)` gör också att värdet skrivs ut.

Ordningen på instruktioner är viktig, så beräkningen kan inte göras innan variablerna till höger om `=` är definierade.

Exemplet ovan kan skrivas tydligare! Variablerna vi använde kanske fungerar bra för den som är van vid att just *s*, *t* och *v* används för sträcka, tid, respektive hastighet, men i de flesta sammanhang kan man inte hoppas på igenkänning av det slaget.

I allmänhet bör man välja variabler som är förklarande, eller till och med 'självdokumenterande'. Det vill säga, man ska inte behöva förklara vad som menas med en variabel utan det ska framgå ändå. Om vi skriver om exemplet ovan med ambitionen att långt senare kunna återkomma till koden och genast förstå vad som menas, så kan det se ut så här:

```
distance = 67
time_used = 1.25
average_velocity = distance / time_used
print(average_velocity)
```

De långa identifierarna kan upplevas som jobbiga att skriva, men erfarenheten är att det i längden lönar sig att skriva kod som är tydlig (eller till och med övertydlig). Om man har valt sina identifierare rätt så blir det ofta lätt att förstå koden.

1.3.1 Kommentarer

Om man vill förtydliga vad en variabel betyder, eller vad som är tänkt att ske i ett komplicerat kodavsnitt, så kan man lägga till kommentarer:

```
distance = 67    # Estimated from map
time_used = 1.25 # Measured by GPS
average_velocity = distance / time_used
print(average_velocity)
```

Tecknet `#` markerar att 'här börjar en kommentar' och allt fram till nästa rad ignoreras helt av Python-tolken. En kommentar kan börja var som helst på en rad.

1.3.2 Fler värden

Vi kan arbeta med olika sorters värden.

```
x = 17
y = 'DA2005'
z = 22/7
q = 22//7    # Operatörn // är för heltalsdivision
```

```
print(x)
print(y)
print(z)
print(q)
print(x, y, z, q)
```

```
17
'DA2005'
3.142857142857143
3
17 DA2005 3.142857142857143 3
```

I exemplet ovan har vi använt följande *typer* av värden:

- x och q är heltal (eng. `int`),
- y är en sträng (eng. `string`), vilket betyder en serie bokstäver,
- z tilldelades kvoten av två heltal och blir då ett *flyttal* (eng. `float`), som är ett sätt att approximera reella tal.

Python har fler primitiva typer vilka vi kommer återkomma till.

Man kan även tilldela flera variabler på en gång:

```
x, y = 2, 3
print(x)
print(y)
```

```
2
3
```

Här används alltså kommatecken för att separera variabler och värden.

Notera att om x och y redan har definierats ovan i koden (t.ex. som 17 och 'DA2005' tidigare) så kommer de att skrivas över med värdena 2 och 3.

1.4 Stränghantering och print

Strängar kan skrivas inom antingen enkelfnuttar ' eller dubbelfnuttar " (jo, programmerare brukar kalla tecknen så istället för apostrof och citattecken!). Man kan på så sätt enkelt blanda ' och ":

```
print("hej")
print('hej')
```

```
hej
hej
```

Eftersom fnuttar kan betyda “start på sträng” och “slut på sträng” kan man inte stoppa in dem hur som helst i strängar, och det är därför som man valt att använda två varianter i Python. Se följande exempel:

```
print("Sa du 'hej'? ")
print('Ja, jag sa "hej".')
```

```
Sa du 'hej'?
Ja, jag sa "hej".
```

Det är rekommenderat att man väljer antingen enkelfnuttar eller dubbelfnuttar som sin standard, och bara använder den andra när det behövs. Så om du valt att använda dubbelfnuttar som din standard så använder du bara enkelfnuttar ifall du ska skriva en sträng som har dubbelfnuttar i sig.

Radbrytningar skrivs med `\n`:

```
print("Hipp\nhipp\nhurra!")
```

```
Hipp
hipp
hurra!
```

Vill man skriva ut flera värden med mellanslag mellan ger man flera argument till `print`:

```
print('hej', 'på', 'dig')
```

```
hej på dig
```

1.5 Aritmetiska operationer

Python stödjer de normala räknesätten:

- addition (+)
- subtraktion (-)
- multiplikation (*)
- division (/)
- heltalsdivision (//)
- exponentiering (**)
- modulus/rest (%)

Prova att *evaluera* följande uttryck:

```
2+3 # ger 5
2-3 # ger -1
2*3 # ger 6
2/3 # ger 0.6666666666666666
2//3 # ger 0
2**3 # ger 8
2%3 # ger 2
```

Dessa fungerar både för heltal och för flyttal. Man kan även använda + på strängar för att sammanfoga dem:

```
'hej' + 'du' # ger 'hejdu'
```

Det läggs dock inte in något mellanslag mellan orden, så det måste man lägga in själv:

```
'hej' + ' ' + 'du' # ger 'hej du'
```

De olika räknesätten binder på förväntat sätt, det vill säga, de vedertagna räknereglerna gäller när man evaluerar aritmetiska uttryck och man behöver inte använda parenteser i onödan. Exempelvis är $2 * 3 + 4$ detsamma som $(2 * 3) + 4$:

```
2 * 3 + 4    # ger 10
(2 * 3) + 4  # ger 10
2 * (3 + 4)  # ger 14
```

I vissa fall kan man använda dem med olika typer av indata på samma gång:

```
2.0 ** 3 - 2.1312    # ger 5.8688
2 * 'hej'            # ger 'hejhej'
```

Men man kan till exempel inte addera en siffra till en sträng eller multiplicera två strängar:

```
'hej' * 'hopp'
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can't multiply sequence by non-int of type 'str'
```

En annan otillåten operation:

```
2 + 'hej'
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Här ser vi exempel på olika felmeddelanden som kan uppstå om man skriver något gale. Felmeddelanden kan ofta vara svårtolkade till en början, men man behöver inte förstå hela felmeddelandet för att få information från det. Om vi bara tittar på sista raden i meddelandet ovan kan vi förstå att felet var att vi hade fel typer på operanderna till operatoren '+', den kan inte hantera kombinationen 'int' och 'str'.

1.6 Input

Vi har sett att print kan användas för att skriva ut värden på skärmen. Funktionen input kan användas för att läsa in data från en användare. Om man ger en sträng som parameter så ges den som "prompt", det vill säga text visar att nu ska man skriva något. Ett exempel:

```
namn = input("Vad heter du?\n")
print("Hej " + namn)
```

Detta lilla program ger i konsolen

```
Vad heter du?
Hej Johan
```

Den första raden skrivs ut av Python, och den andra av den som sitter vid datorn. När man trycker på retur-tangenten returnerar input en sträng med det som användaren skrivit.

1.7 Speciella ord och namn i Python

1.7.1 Reserverade ord

Det finns ett antal *reserverade ord* i Python, alltså ord som tillhör programmeringsspråket:

```
and except lambda with as finally nonlocal while assert False None
yield break for not class from or continue global pass def if raise
del import return elif in True else is try
```

Om man försöker använda ett reserverat ord som variabelnamn får man ett felmeddelande:

```
and = 12
```

```
File "/Users/kxs624/.spyder-py3/DA2005/ht20/f1.py", line 1
```

```
and = 12
    ^
```

```
SyntaxError: invalid syntax
```

Det finns annars ganska få krav på hur namnen kan väljas. Det är dock bra att undvika namn på funktioner som redan finns definierade i python. Det finns exempelvis ingen teknisk begränsning som hindrar dig från att skriva `input = "Hejsan"`, men om du senare vill kalla på funktionen `input` så kommer den inte längre vara tillgänglig.

1.7.2 Identifierare

Identifierare är namn på resurser. Alla variabelnamn är exempel på identifierare, det vill säga, de håller en "resurs" eller ett värde. Det finns dock identifierare som inte är variabler. Till exempel kommer vi gå igenom funktioner, klasser och objekt senare i kursen. Dessa typer namnges med funktionsnamn, klassnamn och objektnamn som alla är identifierare.

Identifierare i Python:

- Kan bestå av bokstäver, siffror, och `_`.
- Får inte börja med siffra.

Python skiljer på versaler och gemener; `Year` och `year` är inte samma identifierare.

Det är viktigt att välja identifierare så att koden blir lätt att läsa. Att döpa en variabel som innehåller någons ålder till `age` är mycket mer informativt än till exempel bara `x`.

1.7.3 Litteraler

En litteral är ett konstant värde som man skriver i programkoden. Till exempel `17`, `2.5`, och `'SU'` är alla litteraler.

Undvik helst att ha litteraler spridda över programmet! Att använda en litteral i ett uttryck kallas ibland för att "hårdkoda" ett värde, och avser det faktum att värdet är fixerat på den platsen i koden. Det är bättre att använda sig av en variabel som definierats på lämplig plats, kanske högst upp i programmet. En variabel är ofta lättare att förstå, givet att man har en lämplig identifierare, än en konstant som dyker upp i programmet. Dessutom så minskar risken för programmeringsfel. Om en konstant används på flera platser i ett program så kanske man inte kommer ihåg att ändra på alla platser om man har anledning att byta värde på konstanten. Om man använder en variabel, som definieras på en rad, så är det endast en plats som man antagligen behöver editera.

1.8 Uppgifter

1. Vad händer om vi kör nedanstående kod och varför blir det som det blir? Undersök!